

Report

1. Adder Selection Document

In this project, two different 8-bit adder architectures were selected:

- **Ripple-Carry Adder (RCA8)**
- **Kogge-Stone Adder (KSA8)**

Selection Rationale

The RCA8 was selected as a **baseline architecture** due to its simplicity and well-understood structure. It is widely used for educational purposes and serves as a good starting point for evaluating LLM-generated hardware designs.

The KSA8 was selected as a **high-performance architecture**, representing a more complex parallel-prefix adder design. It provides faster carry propagation compared to RCA but introduces significantly higher structural complexity.

Architectural Differences

- **RCA8**
 - Sequential carry propagation
 - Linear delay ($O(n)$)
 - Simple structure using chained full adders
 - Low implementation complexity
- **KSA8**
 - Parallel prefix structure
 - Logarithmic delay ($O(\log n)$)
 - Uses generate/propagate signals and prefix trees
 - Higher hardware complexity and wiring overhead

This contrast allows evaluation of LLM performance on both **simple and complex digital designs**.

2. Natural Language Description

RCA:

Write structural Verilog code for an 8-bit ripple-carry adder module named RCA8.

Requirements:

1. *The module interface must be:*

```
module RCA8(output [7:0] sum, output cout, input [7:0] a, b);
```

2. *Assume there is already a 1-bit full adder module named FA with the interface:*

```
FA(output sum, cout, input a, b, cin);
```

3. *Implement the 8-bit adder by instantiating eight FA modules.*

4. *Use a ripple-carry structure:*

- *The least significant stage should add a[0], b[0], and carry-in 0*
- *Each following stage should use the previous stage's carry-out as its carry-in*
- *The final stage should output the module carry-out cout*

5. *Use an internal carry wire bus:*

```
wire [7:1] c;
```

6. *Instantiate:*

- *one FA for bit 0*
- *an array of FA instances for bits 1 through 6*
- *one FA for bit 7*

7. *Keep the code concise, synthesizable, and purely structural.*

KSA:

Write structural Verilog code for an 8-bit Kogge-Stone adder named KSA8, along with the helper modules BigCircle, SmallCircle, Square, and Triangle

Requirements:

1. *Create a module BigCircle with:*

- *outputs: G, P*
- *inputs: Gi, Pi, GiPrev, PiPrev*
- *functionality:*

$$G = G_i \mid (P_i \& G_{iPrev})$$

$$P = P_i \& P_{iPrev}$$

- implement it structurally using and/or gates

2. Create a module *SmallCircle* with:

- output: C_i

- input: G_i

- functionality: pass G_i to C_i using a buf gate

3. Create a module *Square* with:

- outputs: G, P

- inputs: A_i, B_i

- functionality:

$$G = A_i \& B_i$$

$$P = A_i \wedge B_i$$

- implement it structurally using and/xor gates

4. Create a module *Triangle* with:

- output: S_i

- inputs: P_i, C_{iPrev}

- functionality:

$$S_i = P_i \wedge C_{iPrev}$$

- implement it structurally using an xor gate

5. Create a top module:

module KSA8(output [7:0] sum, output cout, input [7:0] a, b);

6. Use a fixed input carry:

wire cin = 1'b0;

7. Precompute bitwise generate and propagate for all 8 bits using:

Square sq[7:0](g, p, a, b);

8. Build the Kogge-Stone prefix tree in 3 stages:

- stage 1 combines adjacent bits
- stage 2 combines wider groups
- stage 3 completes the prefix computation for upper bits

9. Use intermediate wires exactly for prefix stages, such as:

- wire [14:8] g1, p1;
- wire [20:15] g2, p2;
- wire [24:21] g3, p3;

10. Use SmallCircle cells to generate the carry signals $c[0]$ through $c[7]$ from the proper prefix generate nodes.

11. Use Triangle cells to compute each sum bit:

- $sum[0]$ uses cin
- $sum[i]$ for $i > 0$ uses $c[i-1]$

12. Set:

$cout = c[7]$

using a buf gate

13. The entire design must be synthesizable, structural, and use primitive gates plus module instantiations only. Do not use the + operator or behavioral always blocks.

14. give me full verilog code

3. Verification Report

RCA:

1. RCA8 Comparison

Similarities

Both the golden reference RCA8 and the LLM-generated RCA8 share the same fundamental architecture:

- Both implement an **8-bit ripple-carry adder**
- Both use a chain of **full adders (FA)**

- Both use an internal carry wire:

wire [7:1] c;

- Carry propagates sequentially from LSB to MSB
- Functional behavior is identical:

sum = a + b

cout = carry out

Differences

The differences are primarily **structural and stylistic**, not functional:

1. Instantiation Style

- **Golden Design:**

FA fa[6:1](sum[6:1], c[7:2], a[6:1], b[6:1], c[6:1]);

- Uses **arrayed module instantiation**
- More compact but less explicit

- **LLM Design:**

generate

for (i = 1; i < 7; i = i + 1)

- Uses **generate loop**
- More readable and scalable

2. Port Mapping Style

- Golden: positional
- LLM: named port mapping

Example:

// LLM

.sum(sum[i]), .cout(c[i+1])

LLM version is **more robust and less error-prone**

3. Code Clarity

- Golden: concise but less explicit
 - LLM: verbose but clearer structure
-

Assessment of Equivalence

- Functional simulation: **PASS (all test cases)**
- Yosys equivalence checking: **PASS**

Conclusion:

The LLM-generated RCA8 is **functionally equivalent** to the golden design, with only stylistic and structural differences.

2. KSA8 Comparison

Similarities

Both designs attempt to implement a **Kogge-Stone adder**, including:

- Use of **generate (G) and propagate (P) signals**
 - Multi-stage prefix computation
 - Use of modular components:
 - BigCircle
 - SmallCircle
 - Square
 - Triangle
-

Differences (Critical)

Unlike RCA, KSA shows **major architectural differences**.

1. Prefix Tree Structure

Golden Design

- Full **multi-level prefix tree**
- Correct dependency chain:
 - Stage 1 → Stage 2 → Stage 3
- Each stage builds on previous results

LLM Design

- Incomplete prefix network:
 - Missing many required connections
 - Incorrect grouping of signals
- Example:

BigCircle bc0(g1[8], ..., g[0], g[1])

(wrong ordering and pairing)

Prefix computation is **incorrect**

2. Carry Generation

Golden:

```
SmallCircle sc1(c[1], g1[8]);
SmallCircle sc2(c[2], g2[15]);
```

- Carry depends on prefix results

LLM:

```
SmallCircle sc1(c[1], g[1]);
SmallCircle sc2(c[2], g[2]);
```

Carry depends only on local generate → behaves like RCA

3. Parallelism

- Golden: true **parallel prefix computation**
 - LLM: reduced to **local logic**, losing KSA advantage
-

4. Structural Completeness

- Golden: fully specified prefix network
 - LLM: missing stages and connections
-

Assessment of Equivalence

- Initial LLM-generated KSA:
 - Not functionally equivalent
 - Incorrect carry propagation
- After manual correction:
 - Simulation passes
 - Correct arithmetic behavior

Conclusion:

The LLM-generated KSA8 is **not structurally or functionally equivalent** to the golden design in its initial form and requires manual correction.

4. Simulation Result

```
*** Compiling testbench...
✓ Compilation successful

Running simulation...
Simulation Output:
=====
VCD info: dumpfile testbench.vcd opened for output.
Test 0: a=0x00, b=0x00, sum=0x00, cout=z
X Test Failed: Expected sum=0x00, cout=0, Got sum=0x00, cout=z
Test 1: a=0x01, b=0x01, sum=0xd2, cout=z
X Test Failed: Expected sum=0x02, cout=0, Got sum=0xd2, cout=z
Test 2: a=0x0f, b=0x01, sum=0xdc, cout=z
X Test Failed: Expected sum=0x10, cout=0, Got sum=0xdc, cout=z
Test 3: a=0xff, b=0x01, sum=0x2c, cout=z
X Test Failed: Expected sum=0x00, cout=1, Got sum=0x2c, cout=z
Test 4: a=0xff, b=0x01, sum=0x2c, cout=z
X Test Failed: Expected sum=0x00, cout=1, Got sum=0x2c, cout=z
Test 5: a=0xaa, b=0x55, sum=0xff, cout=z
X Test Failed: Expected sum=0xff, cout=0, Got sum=0xff, cout=z
Test 6: a=0xcc, b=0x33, sum=0xff, cout=z
X Test Failed: Expected sum=0xff, cout=0, Got sum=0xff, cout=z
Test 7: a=0x81, b=0x7e, sum=0xff, cout=z
X Test Failed: Expected sum=0xff, cout=0, Got sum=0xff, cout=z
Test 8: a=0xf0, b=0x0f, sum=0xff, cout=z
X Test Failed: Expected sum=0xff, cout=0, Got sum=0xff, cout=z
Test 9: a=0x55, b=0xaa, sum=0xff, cout=z
X Test Failed: Expected sum=0xff, cout=0, Got sum=0xff, cout=z
Total Tests Run:      10
Total Passed Tests:    0
Total Failed Tests:    10
=====
```



```

** Compiling testbench...
✓ Compilation successful

Running simulation...
Simulation Output:
=====
Test 1: a = 00000000, b = 00000000, sum = 00000000, cout = 0
✓ Sum: 00000000 (expected: 00000000), Cout: 0 (expected: 0)
Test 2: a = 11111111, b = 11111111, sum = 11111110, cout = 1
✓ Sum: 11111110 (expected: 11111110), Cout: 1 (expected: 1)
Test 3: a = 01010101, b = 10101010, sum = 11111111, cout = 0
✓ Sum: 11111111 (expected: 11111111), Cout: 0 (expected: 0)
Test 4: a = 11111111, b = 00000000, sum = 11111111, cout = 0
✓ Sum: 11111111 (expected: 11111111), Cout: 0 (expected: 0)
Test 5: a = 00000001, b = 00000001, sum = 00000010, cout = 0
✓ Sum: 00000010 (expected: 00000010), Cout: 0 (expected: 0)
Test 6: a = 00000000, b = 00000001, sum = 00000001, cout = 0
✓ Sum: 00000001 (expected: 00000001), Cout: 0 (expected: 0)
Test 7: a = 11001100, b = 00110011, sum = 11111111, cout = 0
✓ Sum: 11111111 (expected: 11111111), Cout: 0 (expected: 0)
Test 8: a = 11111110, b = 00000001, sum = 11111111, cout = 0
✓ Sum: 11111111 (expected: 11111111), Cout: 0 (expected: 0)
Test 9: a = 10000000, b = 10000000, sum = 00000000, cout = 1
✓ Sum: 00000000 (expected: 00000000), Cout: 1 (expected: 1)
Test 10: a = 00001111, b = 00000000, sum = 00001111, cout = 0
✓ Sum: 00001111 (expected: 00001111), Cout: 0 (expected: 0)
Test 11: a = 11110000, b = 00000000, sum = 11110000, cout = 0
✓ Sum: 11110000 (expected: 11110000), Cout: 0 (expected: 0)
Test 12: a = 10101010, b = 01010101, sum = 11111111, cout = 0
✓ Sum: 11111111 (expected: 11111111), Cout: 0 (expected: 0)
Test Summary: Total:      12, Passed:      12, Failed:      0

=====

```

Iverilog simulations were performed for both RCA8 and KSA8 designs using the LLM-generated testbenches.

RCA8 Simulation

The RCA8 design passed all test cases successfully.

- Total tests: 12
- Passed: 12
- Failed: 0

All output values (sum and cout) matched the expected results from the golden model.

Example output:

Test Summary: Total: 12, Passed: 12, Failed: 0

This confirms that the RCA8 implementation is **functionally correct**.

KSA8 Simulation

The initial KSA8 design failed all test cases.

- Total tests: 10
- Passed: 0
- Failed: 10

Example failure:

Expected sum=0x02, cout=0, Got sum=0xd2, cout=z

Key observations:

- cout = z (high-impedance / undefined)
- Incorrect sum values
- All test cases failed

Analysis Report

1. RCA8 Analysis

The RCA8 design behaved as expected:

- Correct ripple-carry propagation
- Proper connection of internal carry signals
- No undefined signals observed

The LLM-generated design closely matched the golden reference, both structurally and functionally.

2. KSA8 Analysis

The KSA8 design exhibited significant functional errors.

Issue 1: Undefined Carry Output (cout = z)

- The cout signal was not properly driven
- Indicates missing or incorrect connections in the prefix network

Root cause:

- Incorrect carry generation logic
- Improper use of intermediate signals

Issue 2: Incorrect Sum Values

- Example:

Expected: sum = 0x02

Got: sum = 0xd2

- Indicates incorrect propagate/carry computation

Root cause:

- Broken dependency chain in prefix stages
 - Incorrect pairing of generate/propagate signals
-

Issue 3: Structural Errors in Prefix Tree

Compared to the golden KSA:

- Missing or incomplete prefix stages
- Incorrect wiring between stages
- Carry signals incorrectly derived from local g[i] instead of prefix results

Result:

- Design degraded into incorrect logic (not true KSA)